CSE 320 - Computer Networks A

LAB Session 4

13.04.2026

Simple TCP Communication Lab in C

TCP Echo + Message Analyzer Lab

Goal: By the end of the lab, students should be able to:

1. create a TCP socket in C
2. bind and listen on a server
3. connect from a client
4. send and receive data over TCP
5. run both programs from the Windows command line

An example TCP system:

1. **server** waits for a connection
2. **client** connects and sends a message
3. **server** prints the message and replies
4. **client** prints the reply

Server:

- listens on `127.0.0.1:8081`
- accepts one client
- receives a message from client
- prints the message
- counts the number of characters in the message
- sends back a reply in this format:

ECHO: <message> | LENGTH: <number>

Client:

- connects to server
- asks user to type a message
- sends it to server
- receives the server reply
- prints the reply

1. Prepare files

- **`server.c, client.c`**

2. Add required headers

server.c

```c
#include <stdio.h>
#include <winsock2.h>
#pragma comment(lib, "ws2_32.lib")

```

client.c

```c
#include <stdio.h>
#include <winsock2.h>
#pragma comment(lib, "ws2_32.lib")
```

You'll need c socket functions and window's own socket library.

3. Start WinSock

Both programs will require the following

```
WSADATA wsa;
WSAStartup(MAKEWORD(2,2), &wsa);
```

4. Create a TCP Socket

AF_INET IPv4, SOCK_STREAM TCP

```
SOCKET s = socket(AF_INET, SOCK_STREAM, 0);
```

5. Start main(

Create the main functions in both programs (client.c and server.c)

```
5   int main()
6   {
7
8   }
9
```

A. Server Side

1. Declare Variables

```
    WSADATA wsa;
    SOCKET server_socket, client_socket;
    struct sockaddr_in server, client;
    int client_size;
    char buffer[1024] = {0};
    char reply[1200];
```

2. Initialize Winsock

```
4       if (WSAStartup(MAKEWORD(2,2), &wsa) != 0) {
5           printf("WSAStartup failed.\n");
6           return 1;
7       }
```

3. Create Server Socket and Set Server Address

In the server program

```
17      server_socket = socket(AF_INET, SOCK_STREAM, 0);
18      if (server_socket == INVALID_SOCKET) {
19          printf("Could not create socket.\n");
20          WSACleanup();
21          return 1;
22      }
```

```
24      server.sin_family = AF_INET;
25      server.sin_addr.s_addr = inet_addr("127.0.0.1");
26      server.sin_port = htons(8081);
```

4. Server “Bind”

Bind operation that’s specific to the server model

```
28      if (bind(server_socket, (struct sockaddr *)&server,
29               sizeof(server)) == SOCKET_ERROR) {
30          printf("Bind failed.\n");
31          closesocket(server_socket);
32          WSACleanup();
33          return 1;
34      }
```

5. “Listen”

Operations to listen from server side

```
36      listen(server_socket, 1);
37      printf("Server listening on 127.0.0.1:8081...\n");
38
```

6. When client is sending a message, server “Accept”s it

```
39      client_size = sizeof(struct sockaddr_in);
40      client_socket = accept(server_socket, (struct sockaddr *)&client, &client_size);
41
42      if (client_socket == INVALID_SOCKET) {
43          printf("Accept failed.\n");
44          closesocket(server_socket);
45          WSACleanup();
46          return 1;
47      }
```

7. “Receive” message from the Client

```
49      recv(client_socket, buffer, sizeof(buffer), 0);
50      printf("Client says: %s\n", buffer);
51
```

8. Reply Client

```
52      sprintf(reply, "ECHO: %s | LENGTH: %d", buffer, strlen(buffer));
```

9. **Close Existing Sockets for both Server listen and Message**

```
    closesocket(client_socket);
    closesocket(server_socket);
    WSACleanup();

    return 0;
```

B. Client Side

1. **Declare Variables**

```
    WSADATA wsa;
    SOCKET s;
    struct sockaddr_in server;
    char message[1024];
    char server_reply[1200] = {0};
```

2. **Initialize Winsock**

```
    if (WSAStartup(MAKEWORD(2,2), &wsa) != 0) {
        printf("WSAStartup failed.\n");
        return 1;
    }
```

3. **Create Socket**

```
    s = socket(AF_INET, SOCK_STREAM, 0);
    if (s == INVALID_SOCKET) {
        printf("Could not create socket.\n");
        WSACleanup();
        return 1;
    }
```

4. **Set Server Info**

```
    server.sin_family = AF_INET;
    server.sin_addr.s_addr = inet_addr("127.0.0.1");
    server.sin_port = htons(8081);
```

5. **Client “Connect”s**

```
30      if (connect(s, (struct sockaddr *)&server, sizeof(server)) < 0) {
31          printf("Connection failed.\n");
32          closesocket(s);
33          WSACleanup();
34          return 1;
35      }
```

6. **Get message from user and Remove newline from `fgets`**

Removal at the last line

```
37      printf("Enter a message: ");
38      fgets(message, sizeof(message), stdin);
39      message[strcspn(message, "\n")] = '\0';
40
```

7. **Send Message**

```
41      send(s, message, strlen(message), 0);
```

8. **Receive reply and Print**

```
42      recv(s, server_reply, sizeof(server_reply), 0);
43
44      printf("Server replied: %s\n", server_reply);
45
```

9. **Close Socket and terminate**

```
46      closesocket(s);
47      WSACleanup();
48
49      return 0;
```

C. Compile and Run

Visual Studio (on **Developer Command Prompt for Visual Studio**)

cl server.c ws2_32.lib

cl client.c ws2_32.lib

or

Use MinGW gcc compile
https://www.mingw-w64.org/downloads/#msys2
https://www.msys2.org/

In the program file, run the mingw64.exe and type “pacman -S mingw-w64-x86_64-gcc” and type Y for installation
Locate to the code files. E.g. cd /c/Users/manol/lab4 (instead of C:\, /c/..)

gcc server.c -o server -lws2_32
gcc client.c -o client -lws2_32

Open Two terminal windows and run each program with their names
Terminal 1 -> server
Terminal 2 -> client

For MinGW